DOCUMENT

07

Test Plan and Report

Strategy, coverage and why the tests can fail

1. Strategy

Level	Tool	Purpose	Count
Policy (unit)	pytest	Every rule, each rule class on its own, boundaries, and plugging in a new rule	63
Service (unit)	pytest + `FakeInvoiceRepository`	Lifecycle, persistence, refusal side effects, state errors, concurrency, injected policy, clock and store	62
API (integration)	pytest + FastAPI client	Status codes, messages, auth, validation, JSON shape, injected user directory and policy	36
Repository contract	pytest	One contract run against the fake and the in-memory repository (substitutability)	16
UI (end-to-end)	Playwright	A refused over-limit approval seen through the real UI	1
Mutation	mutmut	Whether the unit tests can actually fail	132 mutants

Result of the last run: `python -m pytest backend/tests -q` gives **177 passed**; `npm test` in `e2e/` gives **1 passed**.

2. What is covered

Policy tests (`test_approval_policy.py`)

- Manager allowed up to and including 50,000.00; refused at 50,000.01.
- Director allowed up to and including 500,000.00.
- Roles other than Manager and Director refused for any amount.
- `required_approvals` steps from 1 to 2 only above 500,000.00.
- First approver above 500,000 may be a Manager or a Director.
- Second approver must be a different Director; a Manager second is refused with the Director message.
- Same user refused, including case and whitespace differences, on the new side and the stored side.
- The duplicate-user check wins over the role check.
- Two different Managers are refused above 500,000.
- Role names are matched exactly, so a mistyped role fails closed.
- Limits equal the documented amounts; an approver needs a username.

Service tests (`test_approval_service.py`)

- Exactly at each limit, one cent above and one cent below, through the service.
- First approval keeps the invoice pending with exactly one approval and no decider; second distinct Director completes it.

- Refused attempts (Manager over limit, Admin, same user, Manager second) leave the invoice unchanged.
- A refused attempt is never persisted; first and second approvals are each persisted.
- A refused second attempt does not block a valid second approver afterwards.
- Rejecting a partially approved invoice still works and keeps the recorded approval.
- Wrong state (every status except the allowed one), unknown invoice and reject-without-reason are each refused with exact messages and no update.
- State is checked before permission.
- Two simultaneous approvals by the same user record exactly one; by two Directors both count.

API tests (`test_api.py`)

- 401 without sign-in; bad password rejected; UI served without sign-in.
- 403 with the reason and an unchanged invoice for a Manager over the limit; Admin refusal; Manager second approver.
- Two-Director flow end to end; same Director twice gets 403 and the first approval stays alone.
- Four simultaneous clicks by one Director produce one 200 and three 403.
- Unusable amounts (`1e999999`, NaN, Infinity, sub-cent, negative, zero, too many digits) get 400 and are not stored.
- Amounts at the limits survive the JSON round trip exactly.
- A malformed body gets 400 and leaks no internals.

End-to-end (`e2e/tests/over-limit-rejection.spec.ts`)

Signs in as `demo.manager`, creates and submits its own 75,000.00 invoice, opens Approvals, presses Approve on it, expects the exact refusal text, expects the invoice still listed as pending, then checks the Invoices table still shows Pending Approval. It uses page objects (`LoginPage`, `InvoicesPage`, `ApprovalsPage`) and role-based locators only. Because it creates its own invoice and deletes it afterwards (through the API, as the Admin), it passes whatever the server already holds and leaves it unchanged, so it is repeatable against a server that is already running.

3. Making tests that can fail

Technique	Why
`assert_refused_and_unchanged` helper	Deep copy before; expect `ApprovalNotPermittedError`; compare after; assert the reason text; assert the repository saw no update. An exception-type-only check does not count
Fake repository stores copies and records updates	A no-op `update` let "forgot to save" pass until the mutation run exposed it
`SnapshotRepository` for threads	The first race tests passed even with the lock removed. After the rewrite they fail (2 failed) without the lock and pass with it

Technique	Why
Boundary pairs at every limit	Kills `<` versus `<=` mutants
Exact message assertions	Kills message-text mutants

4. Independent test review

A `test-reviewer` pass judged each test KEEP, STRENGTHEN or REMOVE and reported 14 gaps. They were handled as follows.

Outcome	Items
Strengthened	State-error tests that only checked the exception type; reject-without-reason did not prove nothing changed; the fake hid missing persistence
Removed	2 duplicate tests
Added	Manager-then-manager above the limit, stored-side case and whitespace, 500,000.01 through the service, 50,000.01 by a Director, state-before-permission ordering, role strictness, API 401s and Admin refusal
Declined	Non-positive amounts at policy level (validated at the API boundary instead)
Fixed	The fake `update` ignoring unknown ids

5. Entry and exit criteria

	Criteria
Entry	Spec and tasks exist; tests for a step are written before the code
Exit	All pytest tests pass; mutation score has no unexplained survivors; Playwright test passes; every review finding has a verdict

6. Known gaps

- Only one end-to-end test, as the assignment asks. The two-Director UI flow and the "1 of 2 approvals" text are covered at API level; a second Playwright spec with a reset endpoint is deferred (T024).
- Mutation testing measures the domain unit tests only; API tests are not part of the mutmut run.
- No load or performance tests; the app is a demo with in-memory storage.